

S7 Control Centre — Live End-to-End Demo Guide

Requirement PDF → live AI epic & stories → real GitHub repository → published delivery packs → developer implements with their own AI CLI.

Every step below was executed and verified live on 9 Aug 2026 as run **S7-00043**, producing the real repository `github.com/AlanLands/advisor-portal-signin`.

Part A — One-time preparation

1. Put your model API key in `.env` (e.g. `OPENAI_API_KEY=...`, `LLM_PROVIDER=openai`). Never commit it.
2. Authenticate GitHub CLI: `gh auth status` must show your account logged in.
3. Start the server in **record** mode so every live call is also recorded for deterministic replay on demo day (hard rule 5):

```
lsof -ti :8720 | xargs kill # stop any stale server
LLM_MODE=record ./demo/run_control.sh
```

4. Have the requirement document ready: `demo/MapleSure-Advisor-Portal-Login-Requirement.pdf` — a deliberately small scope (one login page, 7 numbered requirements) so the live build stays demo-sized.

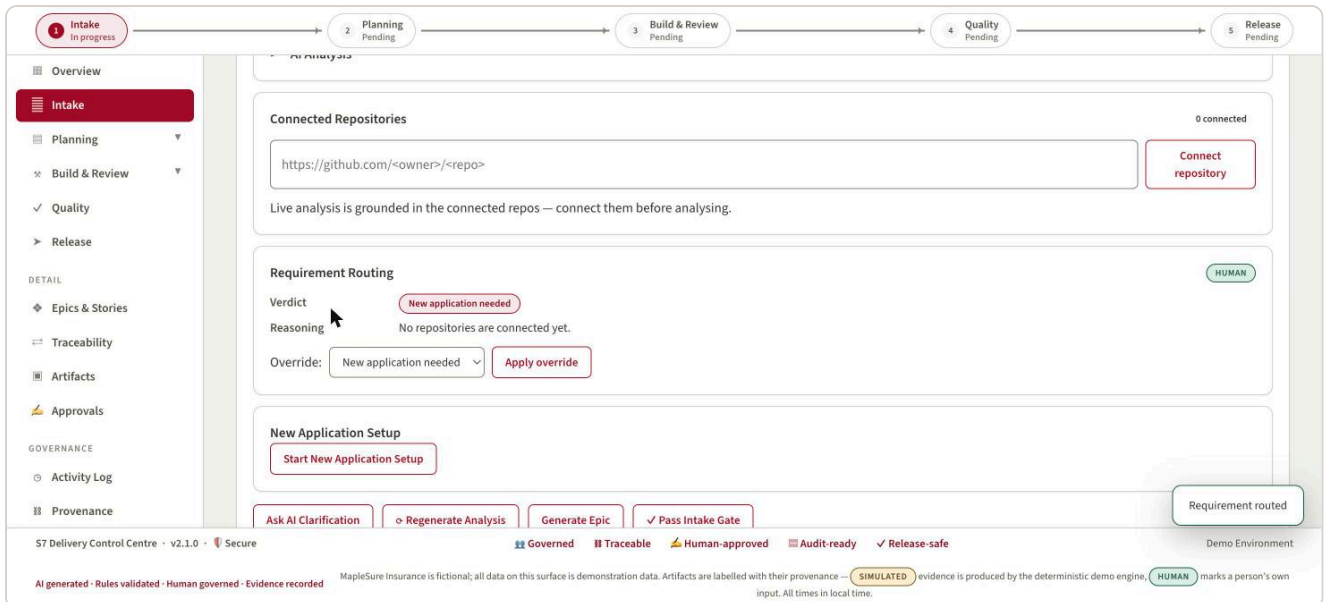
Part B — Intake: PDF in, real repository out VERIFIED LIVE

1. Open `http://127.0.0.1:8720`, create a **new run** and set Environment to **Live**. Set your role to **delivery lead** in the header.
2. Go to **Intake** → **Upload File** → choose the requirement PDF. Live **AI Extraction** runs automatically and returns the epic title, business objective, summary and REQ-01...REQ-07 — badged `LIVE_AI`.

The screenshot displays the S7 Delivery Control Centre interface. At the top, a progress bar shows five stages: 1. Intake (Ready), 2. Planning (Pending), 3. Build & Review (Pending), 4. Quality (Pending), and 5. Release (Pending). The left sidebar contains a navigation menu with sections: Overview, Intake (selected), Planning, Build & Review, Quality, Release, DETAIL (Epics & Stories, Traceability, Artifacts, Approvals), and GOVERNANCE (Activity Log, Provenance). The main content area is divided into three panels. The left panel, titled 'Intake', shows a 'Drag & drop your file here' area with a 'Browse File' button. Below it, it lists supported formats (PDF, DOCX, TXT, MD) and a maximum file size of 10 MB. A file named 'MapleSure-Advisor-Portal-Login-Requirement.pdf' is shown as 'Uploaded'. A note below states: 'Make sure your document contains the business requirement or epic with the scope and key details.' The right panel, titled 'Requirement Summary', contains a text area with the requirement description: 'service staff into internal systems. The business aims to create a new, standalo...'. Below this, 'Extracted Requirements' are listed: REQ-1 (The application must present a sign-in page with an email address field and a password field, reachable at the application root.), REQ-2 (The sign-in form must validate input before submission: both fields are required, and the email address must be well-formed. Validation messages must appear next to the field they concern.), and REQ-3 (The password field must be masked by default and must provide a show/hide toggle.). A 'View Full Extracted Content' link is present. At the bottom of this panel, there are buttons for 'LIVE_AI', 'Edit Extracted Epic', and 'Create Epic & Pass Intake Gate'. The bottom of the interface features a status bar with 'S7 Delivery Control Centre · v2.1.0 · Secure', a row of icons for 'Governed', 'Traceable', 'Human-approved', 'Audit-ready', and 'Release-safe', and the text 'Demo Environment'. A footer note states: 'AI generated · Rules validated · Human governed · Evidence recorded'. A disclaimer below reads: 'MapleSure Insurance is fictional; all data on this surface is demonstration data. Artifacts are labelled with their provenance — SIMULATED evidence is produced by the deterministic demo engine. HUMAN marks a person's own input. All times in local time.'

AI Extraction completed from the uploaded PDF — title, objective, summary and numbered requirements, badged `LIVE_AI`.

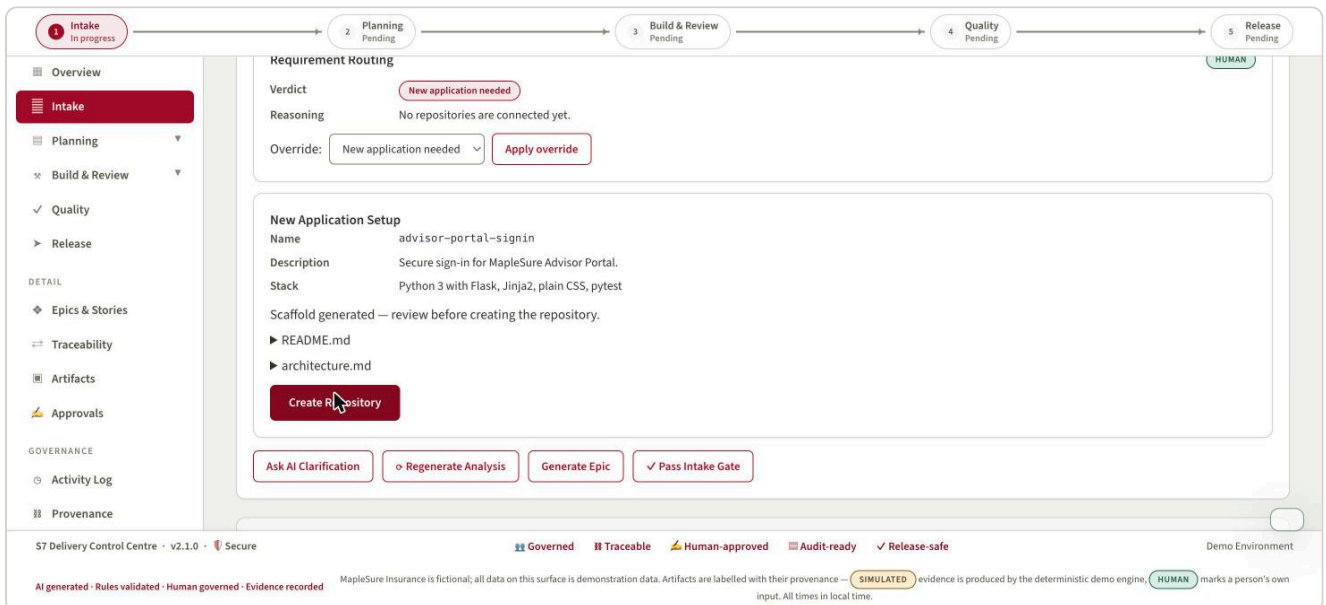
3. Expand **Advanced: Live Analysis & Governance** → click **Route Requirement**. With zero repositories connected the verdict short-circuits to **New application needed** (no model call, 100% confidence). A human override dropdown is available.



Requirement Routing: verdict "New application needed" — reasoning stated, override available.

4. Click **Start New Application Setup**. The model asks up to two rounds of questions — ours asked one: the technology stack. Answer (we used: *Python 3 + Flask, Jinja2, plain CSS, pytest, local seeded JSON credential store*) and click **Submit answers**, then click the setup button again to let it finalize. Result: name `advisor-portal-signin`, description and stack settled.

5. Click **Generate Scaffold** — review the proposed `README.md` and `architecture.md` before anything touches GitHub.



Reviewable scaffold; nothing has been created yet — Create Repository is the explicit approval.

6. Click **Create Repository**. This is the real side effect: `gh repo create --push` creates `github.com/<you>/advisor-portal-signin` and connects it to the run like any other repository.

7. Click **Regenerate Analysis** (live AI, now grounded in the new repo), then **Create Epic & Pass Intake Gate**.

Part C — Planning: live stories and Gate 1

VERIFIED LIVE

- Go to **Planning** → **Generate Plan**. Live AI produced 4 stories against the new repository:

Story	Title	Team
US-1	Create Sign-In Page	Portal Team
US-2	Implement Input Validation	Portal Team
US-3	Authenticate Credentials Locally	Platform Team
US-4	Implement Sign-Out Process	Platform Team

- Review the stories, then **Sign off Gate 1** as business owner. The plan locks at v1 and downstream generation is authorised (never performed) by this approval.

Honesty note: any estimates/story points shown are model estimates, not calibrated from historical delivery data — say so if asked.

Part D — Build & Review: publish governed context to Git

VERIFIED LIVE

- Build & Review** → **Architecture**: click **Generate**. The five-file architecture pack v1 is produced with 9/9 validations passing; click **Accept Architecture** (requires the **engineering lead** role).
- Delivery Packs** → **Generate**: one pack per team (Portal, Platform), layered — task packs reference plan/architecture by version.
- Publish All Packs to Git**. In a live run this writes `AGENTS.md` + `.s7/**` (only those paths) on fresh branches `s7/<run>-<team>` and pushes them to the real repository. Never the default branch; nothing is merged for you. Ours pushed as PUB-001/PUB-002 with `simulated = false`.
- Developer Workspaces**: one workspace per story was provisioned. Click **Assign** on your story and enter the developer's name exactly as they will identify themselves (we used *Alan Lands* on US-1).

The screenshot displays the 'Build & Review' interface, specifically the 'Developer Workspaces' section. The top navigation bar shows a progress flow: Intake (Completed) → Planning (Completed) → Build & Review (Ready) → Quality (Pending) → Release (Pending). The left sidebar lists various sections: Overview, Intake, Planning, Build & Review (selected), Overview, Architecture, Delivery Packs, Developer Workspaces (highlighted), Build & Test Evidence, Independent Review, Build Summary, and Quality. The main content area is titled 'Developer Workspaces' and includes a sub-header 'Human Controlled' and 'AI Assisted'. Below this, a summary row shows: 4 Workspaces Ready, 1 Developers Assigned, 0 Active Development, 0 Open Pull Requests, 0 CI Pipelines Running, and 0 Blocked Workspaces. A search bar and filters for TEAM, DEVELOPER, DEV STATUS, ARTIFACT STATUS, and CI STATUS are present. The main table lists three stories: US-1 (Create Sign-In Page) assigned to Alan Lands, US-2 (Implement Input Validation) assigned to Unassigned, and US-3 (Authenticate Credentials Locally) assigned to Unassigned. Each row shows the repository/branch, delivery pack, latest commit, pull request, CI status, and dev status. The bottom of the interface includes a footer with version information (v2.1.0), security status (Secure), and a disclaimer about simulated data.

US-1 assigned to Alan Lands; unassigned rows keep an Assign action.

- Assignment refreshes the team pack to v2 — **Delivery Packs** now shows **UPDATE PENDING** ("v1 on branch — publish v2 to sync assignments"). Click **Republish to Git**. The branch then carries `.s7/shared/assigned-stories.json` with `"assigned_to": "Alan Lands"`.

Intake
Completed

Planning
Completed

Build & Review
Ready

Quality
Pending

Release
Pending

RUN

Overview

Intake

Planning

Build & Review

Overview

Architecture

Delivery Packs

Developer Workspaces

Build & Test Evidence

Independent Review

Build Summary

Quality

Build & Review · Delivery Packs

Delivery Packs

Governed engineering context generated for each team and ready to publish or download.

Download All Packs

Publish All Packs to Git

2 Teams
All teams covered

2 Delivery Packs
All published

4 Stories Covered
100% of scoped stories

52 Artifacts Linked
Across all packs

25 KB Total Pack Size
Uncompressed

0 Publish Errors
All good

Search by team, repository or stories...

All

All

All

Reset

TEAM	STORIES	REPOSITORY	PACK VERSION	ARTIFACT STATUS	PUBLICATION STATUS	LAST UPDATED	ACTIONS
PT Portal Team 2 stories	US-1, US-2	advisor-portal-signin service	v2.0 arch v1	CURRENT 26 artifacts	PUBLISHED Synced to Git	09 Aug 2026 13:03	Preview Download
PT Platform Team 2 stories	US-3, US-4	advisor-portal-signin service	v1.0 arch v1	CURRENT 26 artifacts	PUBLISHED Synced to Git	09 Aug 2026 13:02	Preview Download

Showing 2 of 2 packs

Selected Pack

Portal Team Delivery Pack v2.0

REPOSITORY

advisor-portal-signin

PUBLICATION STATUS

PUBLISHED

Synced to Git

ARTIFACTS

SIZE

26

13 KB

STORIES INCLUDED

US-1

US-2

PACK CONTENTS

Architecture Reference v1 (by reference)

AGENTS.md (Team Guidance) 1

Stories 2

Acceptance Criteria 4

Task Packs 2

Dependencies 1

Test Strategy 1

S7 Delivery Control Centre · v2.1.0 · Secure

Governed

Traceable

Human-approved

Audit-ready

Release-safe

Demo Environment

AI generated · Rules validated · Human governed · Evidence recorded

MapleSure Insurance is fictional; all data on this surface is demonstration data. Artifacts are labelled with their provenance —

SIMULATED

 evidence is produced by the deterministic demo engine.

HUMAN

 marks a person's own input. All times in local time.

Both packs PUBLISHED — Synced to Git (real push); Portal Team at v2.0 after the assignment republish.

Part E — The developer machine: any AI CLI, no custom plugin

VERIFIED LIVE

On a fresh machine (nothing installed from S7 — this is the point):

1. Clone and check out the published context branch:

```
git clone https://github.com/<you>/advisor-portal-signin.git
cd advisor-portal-signin
git checkout s7/s7-00043-portal-team
```

The clone contains `AGENTS.md` at the root plus the full `.s7/` tree: shared architecture and engineering rules, per-story packs (story.md, acceptance-criteria.md), per-task packs (task.md, context.json, test-plan.md), and `assigned-stories.json`.

2. Open your coding agent in the clone (Claude Code: just run `claude`) — it reads `AGENTS.md` automatically, which tells it exactly how to answer the next question.
3. Ask: *"What user story is assigned to me? I'm Alan Lands."*
The agent matches your name in `.s7/shared/assigned-stories.json` and answers: **US-1 — Create Sign-In Page** (2 acceptance criteria), then walks you into `.s7/stories/US-1/`. If no story carries your name it says so — the instructions forbid guessing.
4. Say: *"Implement it."* The agent builds the Flask login page live, guided by the acceptance criteria and test plan, staying inside the pack's allowed components. Run it and show the page:

```
pip install flask pytest
python app.py          # open http://127.0.0.1:5000 — the live login page
pytest                 # acceptance-criteria tests
```

5. Commit on a working branch — **reference the story id in every commit message** (e.g. `US-1: add sign-in form`; the published `AGENTS.md` tells the agent to do this) — and push:

```
git checkout -b feature/us-1-sign-in
git commit -m "US-1: add sign-in form"
git push -u origin feature/us-1-sign-in
```

6. Back in the Control Centre: **Developer Workspaces** → **Sync from Git**. S7 fetches the repository (read-only) and shows the real progress: the pushed commit sha, the developer branch, how many commits reference the story, and whether the work has been merged to the default branch. Commits are matched by the story/task id in the message; S7's own `s7/**` publication commits never count as developer progress. What git cannot prove (CI, PR state) stays blank rather than invented.

MS

MAPLESURE INSURANCE

S7 Delivery Control Centre

SCENARIOOnline disability claim submission for plan sponsors

RUN ID: S7-00043

IntakeCompleted

PlanningCompleted

Build & ReviewIn progress

RUN

Overview

Intake

Planning

Build & Review

Overview

Architecture

Delivery Packs

Developer Workspaces

Build & Test Evidence

Independent Review

Build Summary

Quality

Release

Build & Review > Developer Workspaces

Developer Workspaces

Human ControlledAI Assisted

Human-controlled engineering workspaces provisioned from approved S7 delivery packs. Developers implement

4Workspaces ReadyReady for developers

1Developers AssignedAcross teams

1Active DevelopmentIn progress

Search by team, story, repository, developer...

TEAMAll

TEAM	STORY / TASK	REPOSITORY / BRANCH	DEVELOPER
PT	US-1 Create Sign-In Page TASK-001	advisor-portal-signin s7/s7-00043-portal-te...	AL Alan Lands 43s ago
PT	US-2 Implement Input Validation TASK-002	advisor-portal-signin s7/s7-00043-portal-te...	Unassigned Assign
PT	US-3 Authenticate Credentials	advisor-portal-signin	Unassigned

S7 Delivery Control Centre · v2.1.0 · Secure

GovernedTraceableHuman

AI generated · Rules validated · Human governed · Evidence recorded

MapleSure Insurance is fictional; all data on this surface is demonstration data. Artifacts are

CI Status

Not Started

Last Synced13:33

Task Pack4

Inherited Context5

Acceptance Criteria2

Dependencies0

Scope Control2

Git Handoff Status

Repository

advisor-portal-signin

Branch

s7/s7-00043-portal-team

Pack Published

v2

Publication Commit

e754dad

Developer Changes

2eb6c2f

Pull Request

-

CI

Not Started

Pushed Commits

1 referencing US-1

Latest Push

2eb6c2f — US-1: start sign-in page — Flask app with login route and form (TASK-001) (Alan Lands)

Developer Branch

feature/us-1-sign-in

Merged to Default

Not yet — merging stays a human action

Real pushed work read from the repository (git fetch, read-only); commits matched by story/task id in the message. Last synced 13:33

After Sync from Git: publication commit vs. developer changes (2eb6c2f), 1 pushed commit referencing US-1 on feature/us-1-sign-in — "Merged to Default: Not yet — merging stays a human action".

7. Open a PR and get it reviewed. **Merging is a human action** — S7 never merges for you, the same discipline as every other gate. After the merge, another Sync from Git flips the workspace to **complete**.

Why this framing wins the room: S7 is the governed control plane — it generated, validated and published the engineering context. The developer's own AI tooling (any agent that reads the open AGENTS.md convention) consumes it. Human Controlled · AI Assisted, with no custom IDE plugin to install or trust.

What is real vs. what is staged in this demo

Real (live)	Staged / simulated (badged in the UI)
PDF extraction, routing, new-app setup, analysis, story planning (all model calls, badged LIVE_AI and recorded for replay)	Test results, CI status, PR state and independent-review execution remain simulation-badged or blank — git alone cannot prove them, so S7 never invents them
GitHub repository creation, branch pushes, assignment sync, and developer progress via Sync from Git — real commits, branch and merge state read from the repository	

Demo-day reliability (hard rule 5): this rehearsal ran with LLM_MODE=record , so all S7-side model calls are now recorded. On demo day, run with LLM_MODE=rep lay to make the control-plane half deterministic, and keep only the developer-side code generation truly live.